

Vault

- 🤖
- Components
 - Server
 - Storage Backend
 - Secret Engine (SE)
 - KV SE
 - KV-v2 SE
 - Transit SE
 - Actual workflow
 - Config
 - Unsealed / Sealed
 - Methods
 - Scenario
 - State
 - Replication & Disaster Recovery (DR)
 - Performance scaling
 - Identity
 - Policy
 - Token
 - Orphan token
 - Token accessor
 - TTL
 - Periodic token
 - Types
 - Lease
 - TTL vs. Max TTL
 - Response Wrapping
 - App Role
 - Vault Secrets Operator (VSO)
- CLI
 - Cheat sheet
 - Init
 - Path / endpoint info
 - Common used path
 - DB SE: Dynamic DB cred mgmt
 - Transit SE
 - KV SE
 - Password Policy
 - HA Cluster for Integrated (Raft) Storage
 - Agent / Proxy

- KV & Identity (Entity & Group)
 - File Structure in Different Environments
-



- Passwords, API keys, DB credentials, certs scattered across config, .env, etc.
- Static, long-lived → dynamic, short-lived creds.

Components

Server

Handle requests, apply policy, mng secrets.

Cluster Setup: Active (Leader): 1; Standby: rest

Storage Backend

Save config, e.g. Encryption key, encrypted secrets, ACL, Tokens, LDAP group policy mapping, e.g. Integrated (Raft) Storage, Consul.

Secret Engine (SE)

Store, gen, encrypt data, e.g. KV, PKI, Transit, Cubbyhole, TOTP, Database.

`vault secrets` sub-cmd: `tune`, `list`, `move`, `enable`, `disable`

KV SE

`vault kv metadata delete` permanently deletes secret of *all ver*; `vault kv destroy` permanently deletes secret of *cur ver*.

ACL rules

```
1 | path "secret/data/dev/team-1/*" {
2 |   capabilities = ["create", "read", "update"]
3 | }
```

Delete policy requires spec setup

- Latest ver

- ```
1 | path "secret/data/dev/team-1/*" {
2 | capabilities = ["delete"]
3 | }
```

- Any ver

- ```
1 | path "secret/delete/dev/team-1/*" {
2 |   capabilities = ["update"]
3 | }
```

- Undelete permission

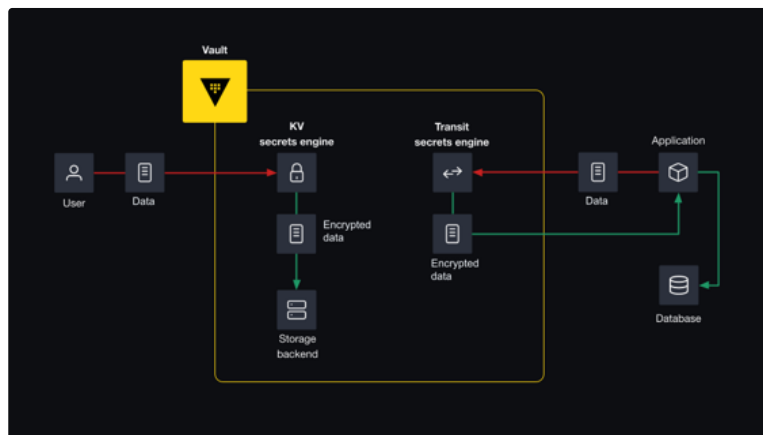
```
1 path "secret/undelete/dev/team-1/*" {
2   capabilities = ["update"]
3 }
```

KV-v2 SE

- Version control: retain configurable #secret ver. This enables data retrieval from older secret ver, in case of undesired data deletion / update.

Transit SE

- so-called EaaS.
- encrypt app DB data, e.g. name, address, ID. Transit key is stored in vault, just like other secrets.
- `vault:v[1,2,3]` #rotation of keys, not ver of encrypted data per se.



Actual workflow

1. Plaintext → Base64 → Ciphertext

```
1 POST $VAULT_ADDR/v1/<mount-path>/encrypt/<key>
2 {
3   "plaintext": <base64-encoded text>
4 }
5
6 echo "{ \"plaintext\": \"$(echo -n 'first-plaintext' | base64)\" }" | \
7 POST -H "X-Vault-Token: $VAULT_TOKEN" \
8   -c "application/json" \
9   $VAULT_ADDR/v1/transit-1/encrypt/crypto
```

HTTP body in `echo`

2. UPSERT email w/ encrypted email

```
1 INSERT INTO users (username, email)
2 VALUES ('bob', 'vault:v1:8n3A...9s==');
3
4 UPDATE users
5 SET email = 'vault:v1:8n3A...9s=='
6 WHERE id = 50;
```

Config

- TTL
-

Unsealed / Sealed

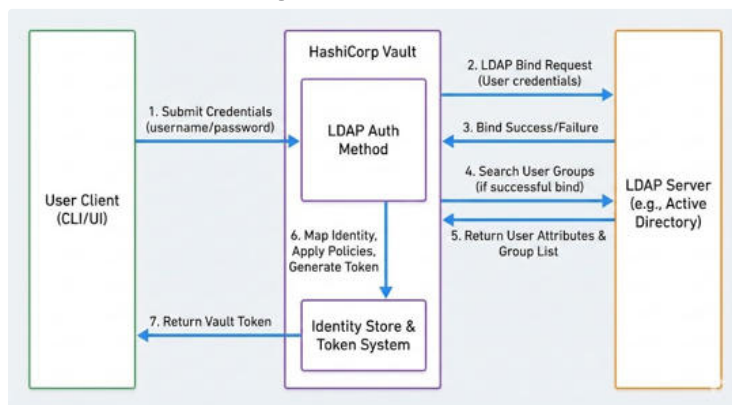
- Shared keys (collect min. requirement) → Master / Root key → Encryption key → Secrets
- For Shamir's method, shared keys distributed among Security, Infra, Senior roles.
- Keyhole, i.e. encryption algo. for encryption key is known to everyone, e.g. AES-256, but reconstruction is almost impossible.

Methods

- Shamir's Secret Sharing: Gen share keys to decrypt Master / Root key

Auth

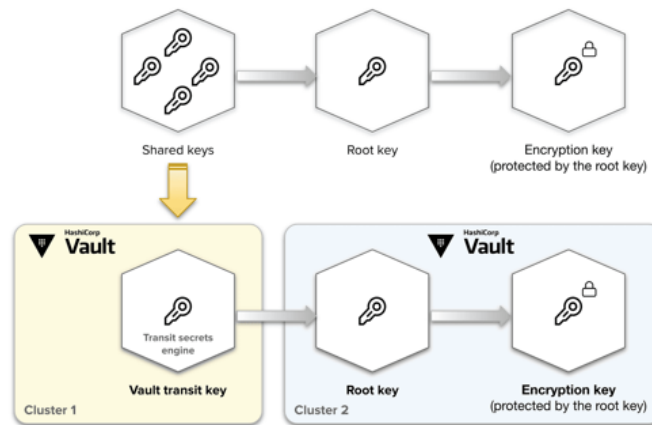
User → Vault [delegate / bind] → LDAP



-
- 1 | `vault operator init -key-shares=7 -key-threshold=4`

(N, K) = (7, 4), N: #shared_keys, K: min. requirement to reconstruct / glue up a master key.

- HSM (Hardware Sec. Mod.) Auto Unseal: Master key → encryption key in HSM → Wrapped key
- Cloud Auto Unseal: Master key → customer-mng key → Wrapped key
 - Customer-mng key is in Cloud provider KMS (Key Mgmt Sys)
- Transit SE



Scenario

Cluster : port mapping : 1: 8200 , 2: 8100

1. Config auto-unseal key provider (Cluster 1)

```
1 vault token create -orphan -policy="autounseal" -wrap-ttl=120 -period=24
```

2. Config auto-unseal (Cluster 2)

Unwrap secret from Cluster 1 & set `VAULT_TOKEN` env var

```
2. 1 vault unwrap -field=token $(cat wrapping-token.txt)
    2 export VAULT_TOKEN="hvs.my_token"
```

`hvs` HashiCorp Vault Service, vide [Types](#).

2. config-autounseal.hcl

```
3. 1 seal "transit" {
    2   address = "$VAULT_ADDR"
    3   disable_renewal = "false"
    4   key_name = "autounseal"
    5   mount_path = "transit/"
    6   tls_skip_verify = "true"
    7 }
```

```
1 vault server -config=config-autounseal.hcl
```

4. Terminate

```
1 pgrep -f vault | xargs kill
2 unset VAULT_TOKEN VAULT_ADDR
3 rm config-autounseal.hcl vault-1.log audit.log
4 rm -r vault/wrapping-token.txt
```

`xargs` : like `grep`, read items from stdin, then build & exec cmds.

i Master key mgmt for Cluster 1? Quis custodiet ipsos custodes? Who will watch the watchmen?

Sol:

- Cluster 1: internal, Cluster 2: external, separated by *Firewall*.
- Or ultimately require Shamir's Secret Sharing, HSM, Cloud auto unseal.

Transit SE auto-unseal

State

- Sealed: Planned maintenance, Power/OS crash, emergency.
 - Master key: DNE, requires shared keys reconstruction / glue up
 - Encryption key: disk storage
- Unsealed: 99% of the time.
 - Master key & Encryption key @server mem

Replication & Disaster Recovery (DR)

2 Vault Cluster Types

- Primary:
 - Automate cluster data snapshots to local / cloud storage
 - Replicate data to Secondary, incl. config, policy, auth methods, SE, tk, audit logs.
- Secondary: cluster maintenance.
 - Performance Secondary: Speed, scaling, locality. Does NOT contain: tk & leases.
 - DR Secondary: backup



- GDPR personally identifiable data not be physically transferred to locations outside EU unless the region / country has an equal rigor of data protection regulation as EU.

Replication → Performance → Secondaries → Paths filter: Deny: EU_GDPR_data/, office_FR/

or `vault secrets enable -local -path=<path/> kv-v2`

Primary & Secondary comm. via Replication Bootstrap Bundle (Secondary actv tk) *qua* secret handshake. Secondary identify Primary by the Bundle and mTLS (mutual TLS) protocol.

P: Performance Cluster qui replicates to a DR Secondary cluster.

D: DR Primary, the cluster being backed-up. Standalone cluster, i.e. w/o Secondary Perf is also D.

$\forall x \in P \implies D, P \subset D$

Performance scaling

- Active node: handle req qui mutate storage.
- Standby node: handle req qui do not mutate storage, e.g. Transit SE (EaaS)

Identity

- Entity: an identity, e.g. a person.
- Group: an identity w/ members, i.e. entities.
- Aliases: entity's accounts w/ auth, e.g. LDAP, GitHub. Must be on different auth mount, otherwise unable to map entity.

Policy

Capabilities: CRUD, list, patch, sudo, deny

```
1 path "auth/userpass/*" {
2   capabilities = [ "create", "read", "update", "delete" ]
3 }
```

1. Define ACL policy w/ HCL (JSON)

```
1 vault policy write <policy-name> <filename>.hcl
2
3 vault policy write developer-vault-policy - << EOF
4 path "dev-secrets/+creds" {
5   capabilities = ["create", "list", "read", "update"]
6 }
7 EOF
```

- + denote any #char.
- Permit CRUD the "creds" path *under any top-lvl path* under dev-secrets/

```
1 path "secret/apps/confidential" {} # confidential per se
2 path "secret/apps/confidential/*" {} # children of confidential
```

Apply policy: `vault token create -policy=<policy-name>`

2. Enable new SE on a path

```
1 vault secrets enable -path=dev-secrets kv-v2
```

3. Get Policy

```
1 vault kv get -output-policy dev-secrets/creds
```

Set user policy

```
1 vault write /auth/userpass/users/danielle-vault-user \
2   password='training' \
3   policies=developer-vault-policy
```

Secret engine

```
1 | vault secrets list
```

Put a secret in kv SE

```
1 | vault kv put /dev-secrets/creds api-key=E6BED968-0FE3-411E-9B9B-C45812E4
```

View secret

```
1 | vault kv get /dev-secrets/creds
```

Login

```
1 | vault login -method=userpass username=danielle-vault-user
```

- Will ask for password, then show tk-policy

Fine-grained ACL param policy

- Param: `required_parameters`, `allowed_parameters`, `denied_parameters`
- Use `allowed` over `denied`, positive setup

```
1 | path "auth/userpass/users/*" {  
2 |   capabilities = ["update"]  
3 |   allowed_parameters = {  
4 |     "password" = []  
5 |   }  
6 | }
```

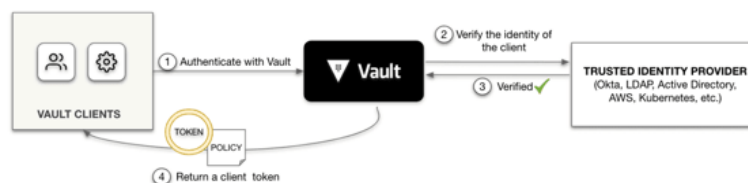
- Allows user update password param val.
- However, cannot update other param val.

Deny: block acc.

```
1 | path "dev-secrets/+ /root" {  
2 |   capabilities = ["deny"]  
3 | }
```

- Dev team `root` acc. not permitted.

Token & Policy: Client (user) auth → Tk-Policy pair



While update is poss, il y a no way to change policies asso. w/ a tk, must *revoke*.

Use Identity to spec user in path


```
1 path "dev-secrets/+/creds/{{identity.entity.name}}" {
2   capabilities = ["create", "list", "read", "update"]
3 }
```

Token

Orphan token

- tk holder creates new tk, new tk will be children of orig. tk
- When parent is revoked, all children revoked as well.
- But orphan can still happen, e.g.
 - a. write `auth/token/create-orphan` endpoint w/ `sudo` permission
 - b. sudo / root acc. `auth/token/create` & set `no_parent=true`
 - c. Tk store roles
 - d. Login w/ non-tk auth method

Token accessor

Tk ref ID, use it to revoke or renew tk.

Turn OFF obfuscation for tk accessor in audit log:

- Pro: Revoke tk immediately, tk accessor plaintext is on the log.
- Con: Hacker got audit logs, and revoke tks by tk accessors in the logs → DoS attack.

P.S. ON: tk accessor is hashed, e.g. `hmac-sha256:7d9s...`

TTL

Set `explicit_max_ttl`

If period/Max TTL not set, TTL will be det. by

1. Sys Max TTL: 32 days.
2. Max TTL on *mount*, override sys Max TTL, i.e. higher priority.
3. Suggested by auth method, config-ed per role, group, user. *SuggestionMaxTTL < MountSysMaxTTL*

▼ Mount

```
"config": { "default_lease_ttl": "1s",
"max_lease_ttl": "5m" }
```

```
"lease_ttl": 0 : use default, i.e. mount or sys
```

Periodic token

Created by

1. sudo / root tk @ `auth/token/create` endpoint.
2. Tk store roles.
3. auth method qui support it, e.g. AppRple.

Only way for an infinte lifetime tk, aside from root tk.

Types

- Service: normal
- Batch: require no storage, limited functionality,
 -  used across Performance Replication clusters, creation scales w/ Performance Standby Node Count
 -  be root tk, create child, renew, manual revoke, be periodic, MaxTTL, have accessors.
- Recovery: TBD

Prefix for type def.

<code>hvs.string</code>	Service
<code>hvb.string</code>	Batch
<code>hvr.string</code>	Recovery

Lease

Time related metadata of Tk & dynamic secrets.

TTL vs. Max TTL

TTL: how frequently app must check-in.

Max TTL: duration a tk can exist, absolute upper limit.

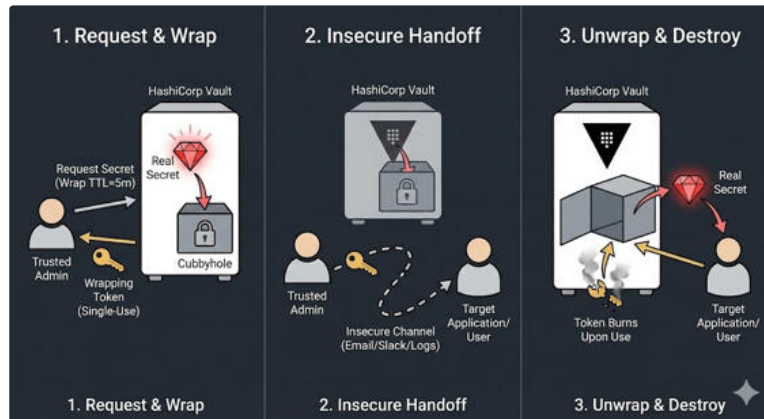
```
1 | vault lease renew/revoke <lease_id>
```

```
1 | # Check all tk
2 | vault list auth/token/accessors
3 |
4 | # Check tk id
5 | vault token lookup -accessor <tk_accessor>
```

Response Wrapping

- Cubbyhole: secret box, a type of SE, each tk has its own storage, cf. kv
- Wrapping Tk: single use for cubbyhole
- Wrap TTL: lifetime of wrapping Tk

Wrapping Tk is intercepted → Real receiver cannot unwrap → Trigger alert



App Role

- RoleID: identifier for a role
- SecretID: dynamic cred, auth-ed for a client to use a RoleID.

Auth w/ Vault-defined roles

```
1 | vault auth enable approle
```

Create a role

```
1 | vault write auth/approle/role/my-role \  
2 |   token_type=batch \  
3 |   secret_id_ttl=10m \  
4 |   token_ttl=20m \  
5 |   token_max_ttl=30m \  
6 |   secret_id_num_uses=40
```

i If tk issued by approle requires to create *child tk*, set `token_num_uses=0`.

Vault Secrets Operator (VSO)

a K8s operator, sync secrets from Vault → K8s, e.g. DB → Vault → K8s VSO ns → other ns

CLI

[CLI Doc: Env var](#)

Cheat sheet

```

1 # Login & Logout
2 export VAULT_TOKEN=<tk>
3 unset VAULT_TOKEN
4
5 # Server off
6 pgrep -f vault | xargs kill
7
8 # List engines / auth method / policies
9 vault <secrets / auth / policy> list
10
11 # List content
12 vault list <path>

```

Init

Dev server: in-mem

```

1 vault server -dev -dev-root-token-id root -dev-tls
2 export VAULT_ADDR=<addr>
3 export VAULT_CACERT='/var/folders/qr/zgzt0sj6n1dxy86s136ntnw0000gn/T/va

```

Prod server

```

1 vault operator init

```

- This will create root tk and shared keys.

Check server status

```

1 vault status -format=json

```

Path / endpoint info

Path Semantics

```

<Mount point>/<Internal
API route>/<Resource
name>

```

isVar

Yes/No/Yes

```

1 vault path-help /auth/userpass

```

Common used path

```

1 secret/data/<name>
2 secret/<metadata>
3
4 transit/keys/<key-name>
5 transit/encrypt/<key-name>
6 transit/decrypt/<key-name>
7 transit/sign/<key-name>
8
9 # Require payload @ HTTP body for auth
10 auth/userpass/login/<username>
11 auth/approle/login
12 auth/kubernetes/login
13 auth/token/create

```

```
14
15 # Dynamic secrets
16 database/creds/<role-name>
17 aws/creds/<role-name>
```

Enable Auth method

```
1 vault auth enable userpass
2 vault auth list
```

View policies

```
1 vault policy list
2
3 vault read sys/policy # raw API acc
```

Token renew & revoke

```
1 vault token renew -accessor <tk-accessor>
2 vault token revoke <tk or tk-accessor>
```

DB SE: Dynamic DB cred mgmt

0. Create role 'class' and spec acc. in DB

```
1 CREATE ROLE \"ro\" NOINHERIT;
2 GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"ro\";
```

1. Set env var for PostgreSQL adr.

SE → DB via plugin interface, in casu `postgresql-database-plugin`

```
1 export POSTGRES_URL=$TF_VAR_POSTGRES_URL
```

or API Gateway (Kong, ngrok) FW adr., do not incl. `tcp://`

2. Enable DB SE

```
1 vault secrets enable database
```

or spec path

```
1 vault secrets enable -path=<mount-path> database
```

`database` : engine type

3. Conn. to DB, config SE

```
1 vault write database/config/postgres \
2   plugin_name=postgresql-database-plugin \
3   connection_url="postgresql://{{username}}:{{password}}@$POSTGRES_URL/
4   allowed_roles=dev \
5   username="root" password="rootpassword"
```

Create dynamic role

1. SQL def. for role instance bind to role class

dev-role.sql

```
1 CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{
2 GRANT ro TO "{{name}}";
```

tee : read stdin, write it to stdout & files. Overwrite file vs. echo "... " >> filename.txt :
append

2. Create role instance (dynamic role)

```
1 vault write database/roles/dev \
2     db_name=postgres \
3     creation_statements=@dev-role.sql \
4     default_ttl=1h \
5     max_ttl=24h
```

3. App / Dev side: req dynamic cred.

```
1 vault read database/creds/dev
```

Chk DB users

```
1 SELECT username, valuntil FROM pg_user;
```

N.B. create pseudo-root user for Vault to utilize rather than using the actual root.

Transit SE

Create encryption keyring orders

```
1 vault write -f transit/keys/orders
```

Keyring: store known encryption keys

Encrypt

```
1 vault write -format=json transit/encrypt/orders \
2     plaintext=$(printf thisIsPlaintext | base64) \
3     | jq -r ".data.ciphertext" > cipher.txt
```

Plaintext must be base64-encoded.

Decrypt

```
1 vault write -field=plaintext transit/decrypt/orders \
2     ciphertext=$(cat cipher.txt) \
3     | base64 -d ; echo
```

KV SE

Write

```
1 vault kv put/patch -mount=secret <secret-name> <k1>=<v1> <k2>=<v2>
```

Mount to `secret/data/my-secret`, could be other path, it is spec in

```
1 vault secrets enable -path=finance-data kv-v2
```

vide [Enable new SE on a path](#), [Enable DB SE](#).

 **PATCH** : update keys; **PUT** : will overwrite all keys

List secrets

```
1 vault kv list -mount=<mount>
```

Read: use `get` for kv engine

```
1 vault kv get -mount=<mount> <secret-name>
2 vault read <mount>/data/<secret-name>
```

Undelete

```
1 vault kv undelete -mount=secret -versions=2 my-secret
```

Permanently Delete

```
1 vault kv destroy -mount=secret -versions=2 my-secret
```

Write Metadata

```
1 vault kv metadata put -custom-metadata=Membership="Platinum" secret/cust
```

Retain #ver

- Global Config

```
1 vault write secret/config max_versions=4
```

- Local Config: overwrite global

```
1 vault kv metadata put -max-versions=4 secret/customer/acme
```

Password Policy

e.g. 6-digit OTP

```
1 length = 6
2 rule "charset" {
3   charset = "0123456789"
4   min-chars = 6
5 }
```

```
1 length = 12
2 rule "charset" {
3   charset = "abcdefghijklmnopqrstuvwxyz"
```

```

4   min-chars = 1
5 }
6 rule "charset" {
7   charset = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
8   min-chars = 1
9 }
10 rule "charset" {
11   charset = "0123456789"
12   min-chars = 1
13 }
14 rule "charset" {
15   charset = "!@#$%^&*()_+!="
16   min-chars = 1
17 }

```

```

1 vault write sys/policies/password/policy-1 policy=my-pwd-policy.hcl
2
3 vault kv put -mount=secret my-secret \
4   password=$(vault read -field password sys/policies/password/policy-1

```

HA Cluster for Integrated (Raft) Storage

`vault_[1,2,3,4]: [Transit auto-unseal, Leader, Follower], Follower` `config-vault_2.hcl`

```

1 storage "raft" {
2   path      = "<path_to_local>/raft-vault_2/"
3   node_id   = "vault_2"
4 }
5
6 # Vault_2 cluster
7 export VAULT_ADDR="http://127.0.0.2:8200"
8
9 vault operator raft list-peers

```

Join nodes

```

1 export VAULT_ADDR="http://127.0.0.3:8200" # adr. for vault_3
2
3 vault operator raft join http://127.0.0.2:8200
4
5 export VAULT_TOKEN=$(cat root-token-vault_2)

```

Retry join: auto join cluster, if node connection details are known `config-vault_4.hcl`

```

1 storage "raft" {
2   path      = "<path_to_local>/raft-vault_4/"
3   node_id   = "vault_4"
4   retry_join {
5     leader_api_addr = "http://127.0.0.2:8200"
6   }
7   retry_join {
8     leader_api_addr = "http://127.0.0.3:8200"
9   }
10 }

```

Resign from Leader (active duty)

```

1 vault operator step-down

```


Remove cluster member

```
1 vault operator raft remove-peer <node-name>
```

Take & Restore SNAPSHOT

```
1 vault operator raft snapshot save <filename>.snapshot
2 vault operator raft snapshot restore
```

Agent / Proxy

Agent > Proxy

Proxy: For static secret caching

agent.hcl

```
1 vault {
2   address = "https://127.0.0.1:8200"
3   tls_skip_verify = true
4 }
5
6 # Auth: Read & mng an existing key
7 auto_auth {
8   method {
9     type = "token_file"
10    config {
11      token_file_path = "/opt/vault/agent-token.txt"
12    }
13  }
14
15  # Sink: Write renewed tk back to the same file.
16  sink "file" {
17    config = {
18      path = "/opt/vault/agent-token.txt"
19      mode = 0600
20    }
21  }
22 }
23
24 # min template to satisfy Agent's validation check.
25 # e.g., IRL: DB login
26 template {
27   source      = "/etc/vault.d/app.ctmpl"
28   destination = "/app/.env"
29   perms       = 0600
30 }
```

/etc/vault.d/app/ctmpl

```
1 # This section queries Vault once for the secret
2 {{ with secret "secret/data/webapp/db" }}
3
4 # .Data.data acc the inner JSON of the KV v2 secret
5 DB_HOST="{{ .Data.data.host }}"
6 DB_PORT="5432"
7 DB_USER="{{ .Data.data.username }}"
8 DB_PASS="{{ .Data.data.password }}"
9
10 # Add logic. If no 'debug' key exists, default to false.
11 DEBUG_MODE="{{ if .Data.data.debug }}{{ .Data.data.debug }}{{ else }}fa
```

```
12 {{ end }}
```

```
1 | vault agent -config=agnet.hcl
```

KV & Identity (Entity & Group)

```
1 vault secrets enable -mount=test-kv -version=2 kv
2
3 vault kv put -mount=test-kv first-secret a=12345 b=23456
4
5 vault kv patch -mount=test-kv first-secret a=aaaaa
6
7 vault kv get -mount=test-kv -version=2 first-secret
8
9 vault kv delete/undelete -mount=test-kv -versions=3,5 first-secret
10
11 # delete/undelete w/ -versions flag, spec by 1,2,3 (w/o space)
12
13
14 # ----- Password gen -----
15 vault write sys/policies/password/policy-1 policy=@/etc/vault.d/6-dgt-o
16
17 vault kv put -mount=test-kv second-secret \
18 password=$(vault read -field password sys/policies/password/policy-1/ge
19
20
21
22 # ----- Write identity policy -----
23 # Obj: Entity ID, userpass accessor ID, Group ID
24
25 vault auth enable userpass
26
27 vault write auth/userpass/users/john password=password policies=default
28
29 vault write identity/entity name="entity-john"
30 # --- Return: John entity ID, e.g. 218b49ce-4271-ae0-c0-57762df2fbb6
31
32 vault auth list -format=json | jq -r '."userpass/".accessor'
33
34 # --- Return: userpass accessor ID, e.g. auth_userpass_5903a8c7
35
36 vault write identity/entity-alias name="john" \
37 canonical_id=<entity_id> \
38 mount_accessor=<userpass_accessor_id>
39
40
41 vault policy write dev /etc/vault.d/dev.hcl
42
43 vault write identity/group \
44 name="managers" \
45 policies=<policy_name> \
46 member_entity_ids=<entity_id>
47
48 # --- Return: Group ID
49
50 vault login -method=userpass username=john
```

File Structure in Different Environments

Machine

Cluster

`/etc/vault.d : config`

`/vault`

`/opt/vault/ : TLS certs`